Labsheet 6

1. server/.env

```
PORT=5000
API_KEY=supersecretkey123
```

2. server/package.json

```
{
  "name": "server",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

3. server/server.js

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');

const app = express();
const PORT = process.env.PORT || 5000;
const API_KEY = process.env.API_KEY;

// Global Middleware
app.use(cors()); // Enable CORS
app.use(express.json());

// Custom Logger Middleware (Method, URL, Timestamp)
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.originalUrl}`);
  next();
});

// Auth Middleware: requires x-api-key header
const checkApiKey = (req, res, next) => {
  const userApiKey = req.headers['x-api-key'];
  if (!userApiKey || userApiKey !== API_KEY) {
    return res.status(401).json({ error: 'Unauthorized: Missing or invalid x-api-key header' });
  }
  next();
};

// In-memory Task list (5 tasks)
let tasks = [
  { id: 1, title: 'Learn Node.js and Express', completed: true },
```

```
  { id: 2, title: 'Build REST API CRUD endpoints', completed: true },
  { id: 3, title: 'Implement Logger and Auth Middleware', completed: false },
  { id: 4, title: 'Consume API using Vanilla JS Fetch', completed: false },
  { id: 5, title: 'Build React Client with Vite', completed: false }
];

// 1. GET /api/tasks
app.get('/api/tasks', (req, res) => {
  res.status(200).json(tasks);
});

// 2. GET /api/tasks/:id
app.get('/api/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id, 10);
  const task = tasks.find(t => t.id === taskId);
  if (!task) {
    return res.status(404).json({ error: `Task with ID ${taskId} not found` });
  }
  res.status(200).json(task);
});

// 3. POST /api/tasks (Protected)
app.post('/api/tasks', checkApiKey, (req, res) => {
  const { title } = req.body;
  if (!title) {
    return res.status(400).json({ error: 'Task title is required' });
  }
  const newTask = {
    id: tasks.length > 0 ? Math.max(...tasks.map(t => t.id)) + 1 : 1,
    title,
    completed: false
  };
  tasks.push(newTask);
  res.status(201).json(newTask);
});

// 4. PUT /api/tasks/:id (Protected)
app.put('/api/tasks/:id', checkApiKey, (req, res) => {
  const taskId = parseInt(req.params.id, 10);
  const task = tasks.find(t => t.id === taskId);
  if (!task) {
    return res.status(404).json({ error: `Task with ID ${taskId} not found` });
  }
  const { title, completed } = req.body;
  if (title !== undefined) task.title = title;
  if (completed !== undefined) task.completed = completed;
  res.status(200).json(task);
});

// 5. DELETE /api/tasks/:id (Protected)
app.delete('/api/tasks/:id', checkApiKey, (req, res) => {
  const taskId = parseInt(req.params.id, 10);
  const taskIndex = tasks.findIndex(t => t.id === taskId);
  if (taskIndex === -1) {
    return res.status(404).json({ error: `Task with ID ${taskId} not found` });
  }
  const deletedTask = tasks.splice(taskIndex, 1);
  res.status(200).json({ message: 'Task deleted successfully', task: deletedTask[0] });
});

// 404 Route Handler
app.use((req, res, next) => {
  res.status(404).json({ error: 'Route not found on this server' });
});

// Centralized Error-Handling Middleware (4 params)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
```

```
    status: err.status || 500,
    error: err.message || 'Internal Server Error'
  });
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

4. client/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>API Lab - Plain JS</title>
  <style>
    body { font-family: sans-serif; padding: 24px; }
    #loading { color: blue; font-weight: bold; }
    #error { color: red; font-weight: bold; }
    li { margin: 8px 0; }
  </style>
</head>
<body>
  <h1>Task List (Plain JS)</h1>
  <div id="loading">Loading...</div>
  <div id="error"></div>
  <ul id="task-list"></ul>

  <script src="script.js"></script>
</body>
</html>
```

5. client/script.js

```
const loadingDiv = document.getElementById('loading');
const errorDiv = document.getElementById('error');
const taskListUl = document.getElementById('task-list');

async function fetchTasks() {
  loadingDiv.style.display = 'block';
  errorDiv.textContent = '';
  taskListUl.innerHTML = '';

  try {
    const response = await fetch('http://localhost:5000/api/tasks');
    if (!response.ok) {
      throw new Error(`Server error: ${response.status} ${response.statusText}`);
    }
    const tasks = await response.json();
    loadingDiv.style.display = 'none';

    tasks.forEach(task => {
      const li = document.createElement('li');
      li.textContent = `${task.title} - [${task.completed ? 'Completed' : 'Pending'}]`;
      taskListUl.appendChild(li);
    });
  } catch (err) {
    loadingDiv.style.display = 'none';
    errorDiv.textContent = `Fetch error: ${err.message}. Make sure server is running!`;
  }
}

fetchTasks();
```

6. client-react/.env

```
VITE_API_URL=http://localhost:5000/api/tasks
VITE_API_KEY=supersecretkey123
```

7. client-react/src/App.jsx

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = import.meta.env.VITE_API_URL;
const API_KEY = import.meta.env.VITE_API_KEY;

export default function App() {
  // 3-state pattern (loading, error, data)
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [newTitle, setNewTitle] = useState('');

  /*
   * DIFFERENCE BETWEEN AXIOS AND FETCH:
   * 1. Axios automatically parses JSON into response.data, fetch needs await response.json().
   * 2. Axios rejects promises on HTTP error codes (4xx/5xx), fetch needs manual response.ok check.
   */
  const loadTasks = async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await axios.get(API_URL);
      setTasks(res.data);
    } catch (err) {
      setError(err.response?.data?.error || err.message);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    loadTasks();
  }, []);

  const handleAddTask = async (e) => {
    e.preventDefault();
    if (!newTitle.trim()) return;

    try {
      const res = await axios.post(
        API_URL,
        { title: newTitle },
        {
          headers: {
            'Content-Type': 'application/json',
            'x-api-key': API_KEY
          }
        }
      );
      setTasks(prev => [...prev, res.data]);
      setNewTitle('');
    } catch (err) {
      alert(`Error creating task: ${err.response?.data?.error || err.message}`);
    }
  };

  const handleDeleteTask = async (id) => {
    try {
      await axios.delete(`${API_URL}/${id}`, {
```

```
      headers: { 'x-api-key': API_KEY }
    });
    setTasks(prev => prev.filter(t => t.id !== id));
  } catch (err) {
    alert(`Error deleting task: ${err.response?.data?.error || err.message}`);
  }
};

return (
  <div style={{ padding: '24px', fontFamily: 'sans-serif', maxWidth: '500px' }}>
    <h2>Task Manager Mini App</h2>

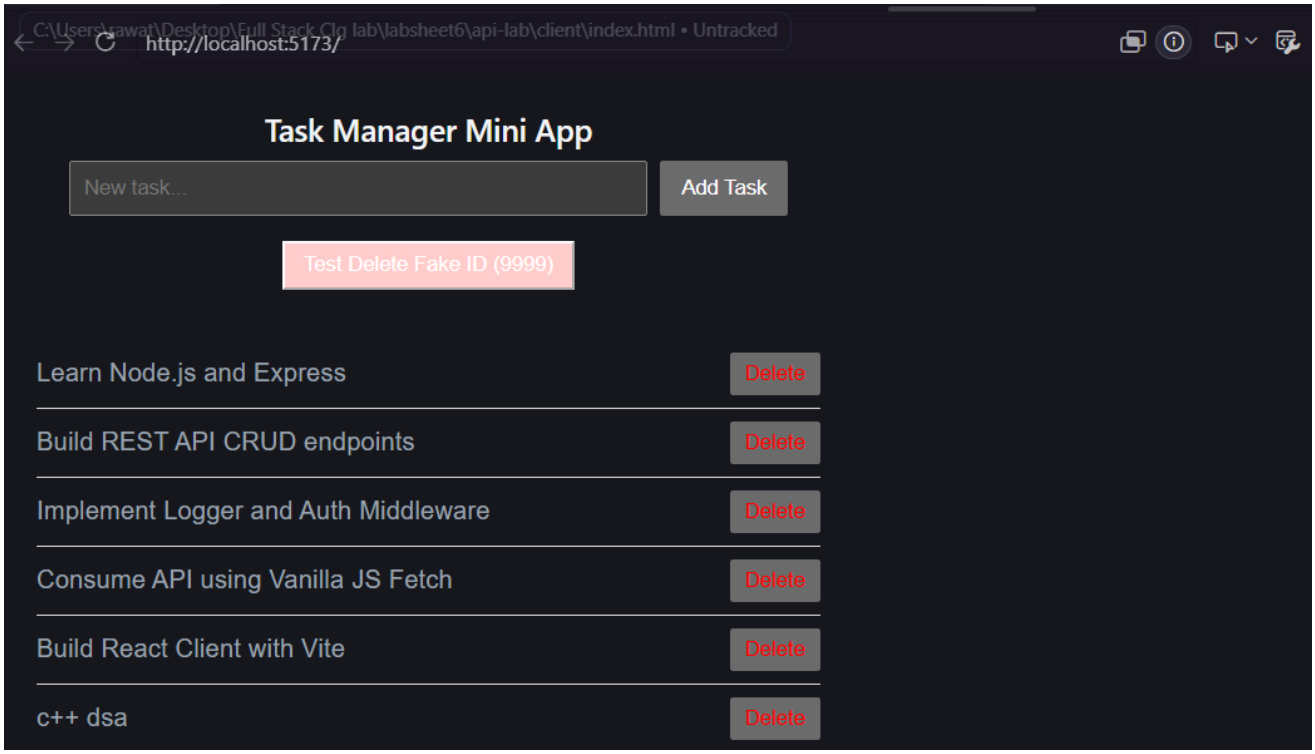
    <form onSubmit={handleAddTask} style={{ marginBottom: '16px' }}>
      <input
        type="text"
        placeholder="New task..."
        value={newTitle}
        onChange={(e) => setNewTitle(e.target.value)}
        style={{ padding: '8px', width: '70%' }}
      />
      <button type="submit" style={{ padding: '8px 12px', marginLeft: '8px' }}>
        Add Task
      </button>
    </form>

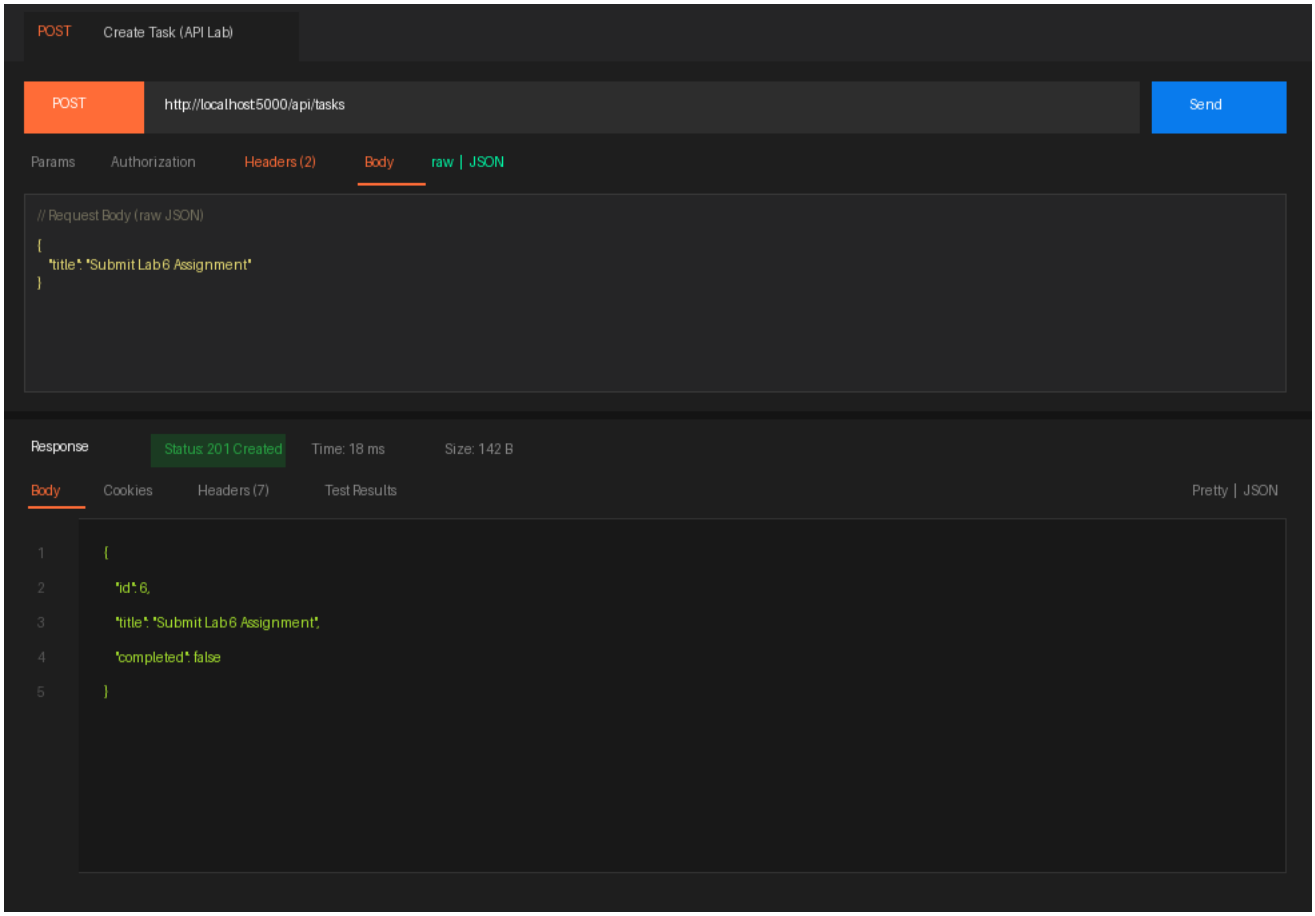
    <button
      onClick={() => handleDeleteTask(9999)}
      style={{ marginBottom: '16px', background: '#ffcccc', padding: '6px 12px' }}
    >
      Test Delete Fake ID (9999)
    </button>

    {loading && <p style={{ color: 'blue' }}>Loading tasks...</p>}
    {error && <p style={{ color: 'red' }}>Error: {error}</p>}

    <ul style={{ listStyle: 'none', padding: 0 }}>
      {tasks.map(task => (
        <li
          key={task.id}
          style={{
            display: 'flex',
            justifyContent: 'space-between',
            alignItems: 'center',
            padding: '8px 0',
            borderBottom: '1px solid #ddd'
          }}
        >
          <span>{task.title}</span>
          <button
            onClick={() => handleDeleteTask(task.id)}
            style={{ color: 'red', cursor: 'pointer', padding: '4px 8px' }}
          >
            Delete
          </button>
        </li>
      ))}
    </ul>
  </div>
);
}
```

Output


C:\Users\rawat\Desktop\Full Stack Clg lab\labsheet6\api-lab\client\index.html • Untracked
http://localhost:5173/
Task Manager Mini App
New task...
Add Task
Test Delete Fake ID (9999)
Learn Node.js and Express
Delete
Build REST API CRUD endpoints
Delete
Implement Logger and Auth Middleware
Delete
Consume API using Vanilla JS Fetch
Delete
Build React Client with Vite
Delete
c++ dsa
Delete


POST
Create Task (API Lab)
POST
http://localhost:5000/api/tasks
Send
Params
Authorization
Headers (2)
Body
raw | JSON
// Request Body (raw JSON)
{
"title": "Submit Lab6 Assignment"
}
Response
Status: 201 Created
Time: 18 ms
Size: 142 B
Body
Cookies
Headers (7)
Test Results
Pretty | JSON
{
"id": 6,
"title": "Submit Lab6 Assignment",
"completed": false
}